



Introdução ao Bun

Manual Técnico de instalação e
guia de uso.

Runtime

SQLite nativo

bun:test

API REST

Docker

SOBRE ESTE MANUAL

Uma introdução prática, sem atalhos conceituais

Bun surgiu como uma tentativa ambiciosa de reduzir a quantidade de ferramentas necessárias no cotidiano de projetos JavaScript e TypeScript. Em vez de tratar runtime, gerenciador de pacotes, bundler e test runner como peças independentes, o projeto reúne essas funções em uma única distribuição. Este manual apresenta essa proposta a partir de uma perspectiva técnica e prática: primeiro discute o runtime e suas diferenças em relação a Node.js e Deno; depois percorre a instalação em Windows, Linux e macOS; por fim, constrói uma pequena API REST com SQLite nativo, testes e containerização.

A edição foi revisada com base na documentação disponível em 16 de agosto de 2026. Na data da revisão, o site oficial distribuía Bun 1.3.14. Como o projeto evolui rapidamente, comandos e APIs experimentais devem sempre ser conferidos na documentação oficial antes de adoção em produção.

Escopo didático

O projeto de `ToDo` foi deliberadamente mantido pequeno. A intenção é deixar visíveis as APIs nativas do Bun — `Bun.serve`, `bun:sqlite` e `bun:test` — sem esconder o funcionamento atrás de frameworks, ORMs ou bibliotecas de validação.

O que você vai construir

Uma API REST em TypeScript executada diretamente pelo Bun.

Persistência local em SQLite por meio do módulo nativo `bun:sqlite`.

Uma organização MVC simples, na qual a view corresponde à representação JSON.

Testes de integração com `bun:test` e banco em memória.

Uma imagem Docker executável com volume persistente para o banco SQLite.

SUMÁRIO

Conteúdo

- 01 Bun: runtime e toolkit**
- 02 Instalação no Windows, Linux e macOS**
- 03 Primeiros passos com a CLI**
- 04 API ToDo com Bun + SQLite**
- 05 Testes: cURL e bun:test**
- 06 Deploy simples com Docker**
- 07 Próximos passos**
- 08 Referências**

CAPÍTULO 01

Bun: runtime e toolkit

De um runtime alternativo a uma plataforma integrada para JavaScript e TypeScript.

1.1 O que é Bun?

Bun é um runtime e um conjunto integrado de ferramentas para JavaScript e TypeScript. A distribuição é entregue como um executável único e reúne quatro frentes que, em muitos projetos Node.js, aparecem como ferramentas distintas: execução de código, instalação de dependências, testes e bundling. A proposta não é criar uma linguagem nova, mas oferecer uma implementação alternativa do ambiente de execução e do toolchain.

A compatibilidade com o ecossistema Node.js é parte central dessa estratégia. O projeto procura executar pacotes publicados no npm, implementa APIs de Node e preserva padrões familiares, como package.json, scripts e node_modules. Isso permite adotar o Bun de forma incremental: uma equipe pode começar apenas por `bun install` ou `bun test` e, posteriormente, avaliar o runtime completo.

Bun como toolkit integrado

Uma única CLI reúne tarefas que normalmente dependem de ferramentas separadas.

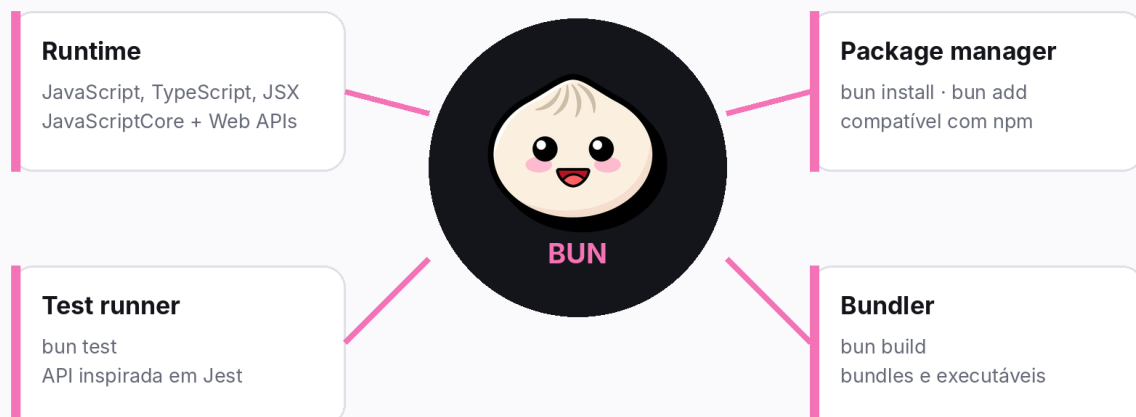


Figura 1 — As quatro funções centrais reunidas pela CLI do Bun. Elaboração própria com base na documentação oficial.

1.2 JavaScriptCore, Web APIs e implementação nativa

A diferença arquitetural mais conhecida está na engine JavaScript. Node.js e Deno utilizam V8; Bun utiliza JavaScriptCore, a engine desenvolvida no ecossistema WebKit e usada pelo Safari. Em torno dessa engine, o Bun implementa APIs de servidor, compatibilidade com Node.js e Web APIs como Request, Response, fetch e URL. Para aplicações HTTP, isso aproxima o código de padrões já familiares ao desenvolvimento web.

Há também uma mudança histórica importante. Bun ficou conhecido, em seus primeiros anos, como um projeto fortemente associado à linguagem Zig. Em julho de 2026, Jarred Sumner publicou o processo de reescrita do Bun em Rust; a documentação atual já descreve o transpiler e o runtime como escritos em Rust. O JavaScriptCore continua sendo a engine responsável por executar JavaScript.

1.3 Quem está por trás do projeto?

Bun foi criado por Jarred Sumner e ganhou forma empresarial sob a Oven. O projeto lançou sua versão 0.1 em 2022 e chegou à versão 1.0 em setembro de 2023. Em dezembro de 2025, a equipe anunciou a aquisição pela Anthropic. O anúncio oficial afirma que o mesmo time continuaria trabalhando no Bun, que o desenvolvimento permaneceria público no GitHub e que a licença MIT seria mantida. Em 2026, o próprio site do Bun já se apresenta como parte da Anthropic.

Por que isso importa?

A aquisição não transforma Bun em uma tecnologia fechada. Para quem avalia adoção, os pontos relevantes são a continuidade da licença MIT, a manutenção pública e o incentivo da Anthropic em usar Bun como infraestrutura para ferramentas de desenvolvimento.

1.4 Bun, Node.js e Deno: diferenças práticas

Aspecto	Bun	Node.js	Deno
Engine	JavaScriptCore	V8	V8
Posicionamento	Runtime + package manager + test runner + bundler integrados	Runtime maduro com ecossistema e governança consolidados	Runtime integrado com foco histórico em Web APIs e segurança por padrão
TypeScript	Executa .ts/.tsx diretamente, com transpile nativo	Suporte nativo leve por type stripping; suporte completo pode usar ferramentas externas	TypeScript é recurso de primeira classe do runtime
Pacotes	bun install; compatível com npm/package.json	npm é o fluxo tradicional; amplo ecossistema npm	Compatível com npm e também com JSR
Testes	bun test, API semelhante a Jest	node:test embutido; Jest/Vitest continuam comuns	deno test embutido
Segurança padrão	Modelo convencional do processo; endurecimento específico no package manager	Modelo convencional do processo	Permissões explícitas e secure defaults
Compatibilidade Node	Objetivo declarado de alta/total compatibilidade	Referência nativa	Compatibilidade ampla com Node e npm

Não existe um vencedor universal nessa comparação. Node.js continua sendo a referência de compatibilidade e maturidade operacional; Deno possui uma filosofia particularmente forte de segurança por padrão e tooling integrado; Bun privilegia integração e desempenho, mantendo a migração a partir do ecossistema Node como objetivo explícito. Para projetos existentes, a compatibilidade com dependências nativas e comportamentos específicos de Node deve ser validada antes da troca de runtime.

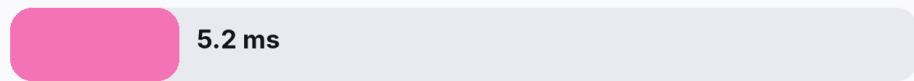
1.5 Sobre benchmarks

A documentação do Bun publica microbenchmarks de inicialização e de subsistemas específicos. Eles são úteis para compreender onde o projeto concentra otimizações, mas não devem ser extrapolados automaticamente para uma API real, que envolve rede, serialização, banco de dados e bibliotecas de terceiros. O gráfico abaixo reproduz o exemplo de startup da documentação do runtime.

Tempo de inicialização em um microbenchmark

Hello World em Linux, valores publicados na documentação oficial do Bun.

Bun



5.2 ms

Node.js



25.1 ms

Leitura correta: mede startup de um script mínimo; não substitui benchmark da sua aplicação.

Figura 2 — Microbenchmark de startup publicado pela documentação do Bun: 5,2 ms para Bun e 25,1 ms para Node.js em um script Hello World no Linux.

CAPÍTULO 02

Instalação no Windows, Linux e macOS

Instalar o Bun é simples; compreender o PATH, os requisitos do sistema e as alternativas de pacote evita problemas depois.

Fluxo recomendado de instalação



Figura 3 — Sequência recomendada de instalação e verificação.

2.1 Antes de instalar

Bun é distribuído como um executável único e sem dependências de runtime adicionais. No Linux, o instalador oficial usa unzip durante a instalação; a documentação recomenda kernel 5.6 ou superior, embora versões tão antigas quanto 3.10 possam funcionar com degradação de syscalls mais novas. No Windows, a versão mínima informada é Windows 10 1809. Para máquinas x64 antigas, existem builds baseline destinados a processadores sem AVX2; o instalador escolhe a variante adequada automaticamente.

Verificação recomendada

Depois de instalar, abra um novo terminal e execute `bun --version` e `bun --revision`. O segundo comando identifica também o commit preciso usado na build.

2.2 macOS

No macOS, o caminho mais direto é o script oficial. Ele detecta a arquitetura da máquina — incluindo Apple Silicon e Intel —, baixa o binário adequado e configura o diretório de instalação no perfil do usuário.

Instalação pelo script oficial

```
curl -fsSL https://bun.com/install | bash

# Feche e reabra o terminal e valide:
bun --version
```

```
bun --revision
```

Se você já usa Homebrew para administrar ferramentas de desenvolvimento, também pode instalar o Bun pelo tap oficial. A vantagem é manter atualização e remoção no mesmo fluxo utilizado para outros pacotes do sistema.

Alternativa via Homebrew

```
brew install oven-sh/bun/bun  
  
# Atualização quando instalado pelo Homebrew:  
brew upgrade bun
```

2.3 Linux

Em distribuições Linux, verifique primeiro se unzip está disponível. Em sistemas Debian/Ubuntu, o pacote pode ser instalado com apt. Depois disso, o script oficial segue o mesmo formato usado no macOS.

Exemplo para Debian/Ubuntu

```
sudo apt update  
sudo apt install -y unzip  
  
curl -fsSL https://bun.com/install | bash  
  
bun --version  
uname -r
```

Bun oferece binários para x64 e ARM64. A documentação também disponibiliza builds musl, úteis em distribuições como Alpine Linux, e informa que os binários glibc requerem glibc 2.17 ou mais recente. Em cenários corporativos com máquinas antigas, a existência das builds baseline pode ser relevante para evitar erros do tipo “Illegal instruction”.

2.4 Windows

O suporte nativo ao Windows passou a fazer parte das versões estáveis a partir do Bun 1.1. Atualmente, a instalação recomendada pode ser feita diretamente no PowerShell. Execute o terminal com uma política compatível com scripts corporativos e use o instalador oficial:

Instalação no PowerShell

```
powershell -c "irm bun.sh/install.ps1|iex"
```

Como alternativa, o Scoop também é listado pela documentação oficial. Essa opção é adequada para quem já centraliza ferramentas de linha de comando por esse gerenciador.

Alternativa via Scoop

```
scoop install bun  
  
# Atualização:  
scoop update bun
```

2.5 Quando bun não é encontrado: corrigindo o PATH

O erro mais comum depois da instalação não é uma falha do runtime, mas um terminal que ainda não conhece o diretório do executável. Em macOS e Linux, o diretório padrão é ~/.bun/bin. Se o instalador não conseguiu persistir a configuração, adicione BUN_INSTALL e PATH ao arquivo de inicialização do seu shell.

PATH em macOS/Linux

```
export BUN_INSTALL="$HOME/.bun"
export PATH="$BUN_INSTALL/bin:$PATH"

# Recarregue o shell (exemplo zsh):
source ~/.zshrc

bun --version
```

No Windows, a instalação padrão fica sob o perfil do usuário. A própria documentação sugere testar o executável diretamente e, se ele funcionar, adicionar o diretório ao PATH do usuário.

Diagnóstico e correção do PATH no Windows

```
& "$env:USERPROFILE\.bun\bin\bun" --version

[System.Environment]::SetEnvironmentVariable(
    "Path",
    [System.Environment]::GetEnvironmentVariable("Path", "User") +
    ";$env:USERPROFILE\.bun\bin",
    [System.EnvironmentVariableTarget]::User
)
```

2.6 Atualização e versão fixa

Instalações feitas pelo script oficial podem ser atualizadas pelo próprio executável. Em ambientes de desenvolvimento é conveniente acompanhar a linha estável; em CI e produção, fixar uma versão reduz surpresas e facilita reproduzir builds.

Atualização e pin de versão

```
# Atualizar para a versão estável mais recente
bun upgrade

# Voltar de canary para stable
bun upgrade --stable

# Exemplo de instalação fixa no macOS/Linux
curl -fsSL https://bun.com/install | bash -s "bun-v1.3.14"
```

CAPÍTULO 03

Primeiros passos com a CLI

Antes de construir a API, vale compreender o ciclo básico de um projeto Bun.

3.1 Criando o projeto

Crie uma pasta vazia e inicialize o projeto. O comando `bun init` gera os arquivos básicos e pode adicionar tipos para o runtime. Para este manual, usaremos TypeScript desde o início.

Inicialização do projeto

```
mkdir bun-todo-api
cd bun-todo-api
bun init -y
```

Em seguida, substitua ou ajuste o `package.json` para manter apenas os scripts necessários ao exemplo. O runtime executa arquivos `.ts` diretamente; não haverá etapa de transpile separada durante o desenvolvimento.

`package.json`

```
{
  "name": "bun-todo-api",
  "type": "module",
  "scripts": {
    "dev": "bun --watch src/server.ts",
    "start": "bun src/server.ts",
    "test": "bun test",
    "test:coverage": "bun test --coverage"
  },
  "devDependencies": {
    "@types/bun": "latest"
  }
}
```

Instalação e execução

```
bun install

# Execute um arquivo TypeScript diretamente:
bun src/server.ts

# Ou um script do package.json:
bun run dev
```

3.2 O que a CLI substitui no fluxo cotidiano

Tarefa	Fluxo comum no ecossistema Node	Bun
Instalar dependências	<code>npm install</code>	<code>bun install</code>
Adicionar pacote	<code>npm install zod</code>	<code>bun add zod</code>
Executar script	<code>npm run dev</code>	<code>bun run dev</code>
Executar pacote temporário	<code>npx <pacote></code>	<code>bunx <pacote></code>
Testes	Jest/Vitest ou <code>node --test</code>	<code>bun test</code>

Tarefa	Fluxo comum no ecossistema Node	Bun
Build/bundle	esbuild/rollup/etc.	bun build

3.3 Segurança durante bun install

Uma diferença operacional importante está nos lifecycle scripts de dependências. Por razões de segurança, Bun não executa automaticamente scripts arbitrários como postinstall para pacotes instalados. Quando um pacote realmente precisa desse comportamento, ele pode ser incluído explicitamente em `trustedDependencies`. Essa política reduz uma superfície conhecida de ataques à cadeia de suprimentos, mas também significa que algumas dependências exigem configuração adicional.

Exemplo de `trustedDependencies`

```
{
  "trustedDependencies": ["pacote-que-precisa-de-postinstall"]
}
```

3.4 Primeiro servidor HTTP

Bun expõe `Bun.serve` como API nativa para servidores HTTP. A interface trabalha diretamente com `Request` e `Response`, seguindo o modelo das Web APIs. Antes do MVC, o menor servidor útil pode ser escrito em poucas linhas:

Servidor HTTP mínimo

```
const server = Bun.serve({
  port: 3000,
  fetch(request) {
    return Response.json({ message: "Olá, Bun!" });
  },
});

console.log(`http://localhost:${server.port}`);
```

CAPÍTULO 04

API ToDo com Bun + SQLite

Uma implementação pequena o suficiente para ser lida inteira, mas organizada em camadas claras.

4.1 Requisitos e decisões de projeto

A API terá cinco operações: listar tarefas, buscar uma tarefa por id, criar, atualizar parcialmente e remover. Não haverá framework HTTP nem ORM. A persistência usa bun:sqlite, o driver SQLite embutido no Bun. A API desse módulo é síncrona, oferece prepared statements, parâmetros, transações e suporte a WAL. Para uma aplicação didática de baixa complexidade, isso reduz dependências e torna o caminho da requisição fácil de acompanhar.

Arquitetura MVC da API de ToDo

A "view" não renderiza HTML: ela transforma o resultado em uma resposta JSON.

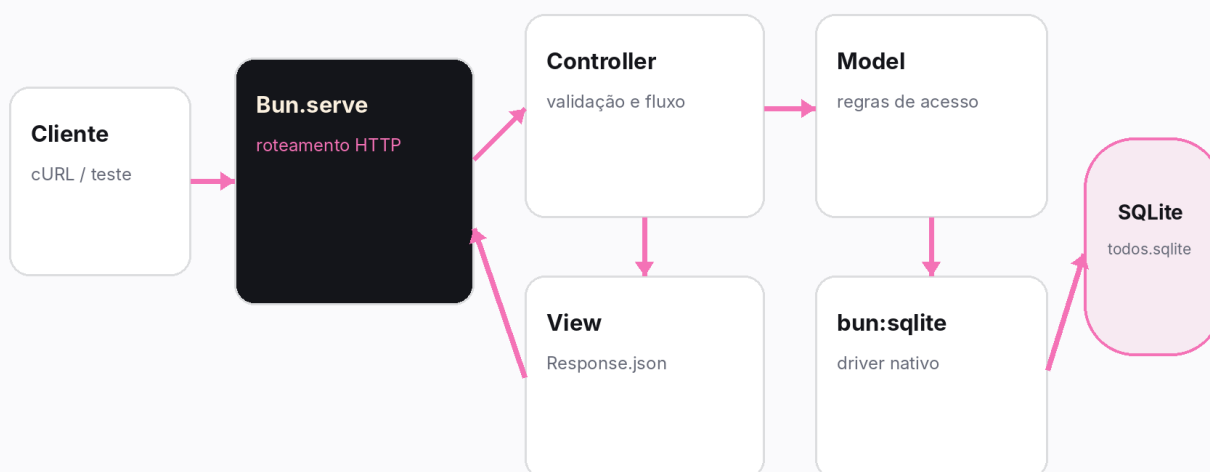


Figura 4 — Fluxo MVC adotado no exemplo. A view é a serialização JSON enviada ao cliente.

4.2 Estrutura de diretórios

Estrutura final

```
bun-todo-api/
├── src/
│   ├── controllers/
│   │   └── TodoController.ts
│   ├── models/
│   │   └── TodoModel.ts
│   ├── views/
│   │   └── jsonView.ts
│   ├── app.ts
│   └── server.ts
├── test/
│   └── api.test.ts
├── data/
│   └── todos.sqlite # criado em runtime
├── package.json
└── bun.lock
```

```
└─ Dockerfile
└─ .dockerignore
```

4.3 Model: acesso ao SQLite

O model abre o banco, cria a tabela quando necessário e encapsula as consultas. Quando o banco é persistente, o exemplo habilita WAL. A documentação do Bun recomenda esse modo para a maioria das aplicações por melhorar a convivência entre leitores concorrentes e um escritor. Em testes, a classe aceita uma instância de Database injetada, o que permite usar `:memory:` sem tocar no sistema de arquivos.

Para fins didáticos, o arquivo foi quebrado em blocos menores. Essa leitura por responsabilidade facilita a compreensão do que pertence à infraestrutura do banco e do que pertence às operações de domínio.

src/models/ToDoModel.ts — tipos, inicialização e abertura do banco

```
import { Database } from "bun:sqlite";
import { mkdirSync } from "node:fs";
import { dirname } from "node:path";

export type Todo = {
  id: number;
  title: string;
  completed: boolean;
  createdAt: string;
};

type TodoRow = {
  id: number;
  title: string;
  completed: number;
  created_at: string;
};

function toTodo(row: TodoRow): Todo {
  return {
    id: row.id,
    title: row.title,
    completed: Boolean(row.completed),
    createdAt: row.created_at,
  };
}

export class ToDoModel {
  constructor(private readonly db: Database = ToDoModel.openDatabase()) {
    this.db.run(`
      CREATE TABLE IF NOT EXISTS todos (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        title TEXT NOT NULL,
        completed INTEGER NOT NULL DEFAULT 0,
        created_at TEXT NOT NULL DEFAULT
          (strftime('%Y-%m-%dT%H:%M:%fZ', 'now'))
      );
    `);
  }

  private static openDatabase(): Database {
    const path = process.env.DB_PATH ?? "data/todos.sqlite";
    if (path !== ":memory:") mkdirSync(dirname(path), { recursive: true });

    const db = new Database(path, { create: true });
    if (path !== ":memory:") db.run("PRAGMA journal_mode = WAL;");
    return db;
  }
}
```

O primeiro bloco define tipos, converte a linha do banco em um objeto da aplicação e prepara a abertura do SQLite com criação automática do diretório e ativação de WAL quando o banco não está em memória.

src/models/ToDoModel.ts — leitura e busca por id

```
list(): Todo[] {
  const rows = this.db
    .query(`SELECT id, title, completed, created_at
           FROM todos ORDER BY id DESC`)
    .all() as TodoRow[];
  return rows.map(toTodo);
}

find(id: number): Todo | null {
  const row = this.db
    .query(`SELECT id, title, completed, created_at
           FROM todos WHERE id = ?`)
    .get(id) as TodoRow | null;
  return row ? toTodo(row) : null;
}
```

Esses métodos concentram as leituras. `list` retorna a coleção completa em ordem decrescente de `id`; `find` resolve uma única linha e devolve `null` quando o registro não existe.

`src/models/ToDoModel.ts` — criação e atualização

```
create(title: string): Todo {
  const result = this.db
    .query("INSERT INTO todos (title) VALUES (?)")
    .run(title);
  return this.find(Number(result.lastInsertRowid));
}

update(id: number, title?: string, completed?: boolean): Todo | null {
  this.db.query(`UPDATE todos
                SET title = COALESCE(?, title),
                    completed = COALESCE(?, completed)
                WHERE id = ?`)
    .run(title ?? null, completed === undefined ? null : Number(completed), id);
  return this.find(id);
}
```

Aqui ficam as operações de escrita. `create` insere e reaproveita `find` para devolver o item recém-criado; `update` usa `COALESCE` para permitir alterações parciais sem duplicar lógica SQL.

`src/models/ToDoModel.ts` — remoção e encerramento do banco

```
remove(id: number): boolean {
  const result = this.db.query("DELETE FROM todos WHERE id = ?").run(id);
  return result.changes > 0;
}

close(): void {
  this.db.close();
}
```

O último trecho fecha o ciclo da classe: `remove` devolve apenas um booleano, suficiente para o controller decidir o status HTTP, e `close` oferece um encerramento explícito para cenários como testes.

Por que converter `completed`?

SQLite representa booleanos como valores inteiros. O model mantém esse detalhe dentro da camada de persistência e expõe boolean para o restante da aplicação. Essa pequena transformação evita que a representação do banco “vaze” para o contrato HTTP.

4.4 View: JSON como representação

Em uma API, MVC não exige que a view seja HTML. A responsabilidade da view é representar a saída. Aqui ela padroniza JSON, status HTTP e uma resposta simples de erro. Como o arquivo é curto, a divisão por função ajuda a deixar o papel de cada utilitário bem explícito.

src/views/jsonView.ts — função json

```
export function json(data: unknown, status = 200): Response {
  return Response.json(data, {
    status,
    headers: { "Cache-Control": "no-store" },
  });
}
```

A função json centraliza a serialização da resposta e o código de status. Ela também fixa Cache-Control como no-store, o que é conveniente para testes e para evitar respostas antigas em desenvolvimento.

src/views/jsonView.ts — função error

```
export function error(message: string, status: number): Response {
  return json({ error: message }, status);
}
```

A função error reaproveita a convenção anterior para devolver um payload simples e previsível de falha. Com isso, o controller não precisa repetir Response.json a cada erro de validação.

4.5 Controller: validação e casos de uso

O controller recebe dados já roteados, valida os campos e chama o model. O exemplo evita uma biblioteca de schema propositalmente: as verificações ficam visíveis. Em um sistema maior, ferramentas como Zod, Valibot ou TypeBox podem substituir essa validação manual sem alterar a divisão de responsabilidades.

src/controllers/ToDoController.ts — imports e construtor

```
import { TodoModel } from "../models/TodoModel";
import { error, json } from "../views/jsonView";

export class ToDoController {
  constructor(private readonly model: TodoModel) {}
}
```

O construtor injeta o model e torna a classe testável. Essa dependência explícita é o elo entre a camada HTTP e a camada de persistência.

src/controllers/ToDoController.ts — list e get

```
list(): Response {
  return json(this.model.list());
}

get(id: number): Response {
  const todo = this.model.find(id);
  return todo ? json(todo) : error("ToDo não encontrado", 404);
}
```

list devolve todos os itens; get faz a busca por id e traduz a ausência do registro em HTTP 404. Note como o controller começa a converter operações de banco em contrato HTTP.

src/controllers/ToDoController.ts — create

```
async create(request: Request): Promise<Response> {
```

```
const body = await request.json().catch(() => null) as { title?: unknown } | null;
const title = typeof body?.title === "string" ? body.title.trim() : "";
if (!title) return error("O campo title é obrigatório", 400);
return json(this.model.create(title), 201);
}
```

create lê o corpo JSON, valida o campo title e só então chama o model. Mesmo sem biblioteca externa, já se observa uma regra importante: a API aceita apenas strings não vazias como título.

src/controllers/ToDoController.ts — update

```
async update(request: Request, id: number): Promise<Response> {
  const body = await request.json().catch(() => null) as {
    title?: unknown;
    completed?: unknown;
  } | null;

  if (!body) return error("JSON inválido", 400);
  if (body.title !== undefined &&
    (typeof body.title !== "string" || !body.title.trim())) {
    return error("title deve ser uma string não vazia", 400);
  }
  if (body.completed !== undefined && typeof body.completed !== "boolean") {
    return error("completed deve ser boolean", 400);
  }

  const todo = this.model.update(
    id,
    typeof body.title === "string" ? body.title.trim() : undefined,
    typeof body.completed === "boolean" ? body.completed : undefined,
  );
  return todo ? json(todo) : error("ToDo não encontrado", 404);
}
```

update é o método mais rico do arquivo: ele aceita atualização parcial, valida tipos opcionais e reaproveita o model para persistir a mudança. O retorno segue simples e previsível: 200 em sucesso, 400 para entrada inválida e 404 quando o item não existe.

src/controllers/ToDoController.ts — remove

```
remove(id: number): Response {
  return this.model.remove(id)
    ? new Response(null, { status: 204 })
    : error("ToDo não encontrado", 404);
}
```

remove completa o conjunto de casos de uso. O retorno 204 sem corpo deixa o contrato alinhado ao comportamento típico de APIs REST para exclusão bem-sucedida.

4.6 Roteamento e composição da aplicação

A função createApp monta o controller e devolve um handler compatível com Bun.serve. Mantê-la separada do processo HTTP é útil para testes: uma Request pode ser passada diretamente para a função, sem abrir porta de rede.

src/app.ts — composição e início do handler

```
import { ToDoController } from "../controllers/ToDoController";
import { TodoModel } from "../models/ToDoModel";
import { error, json } from "../views/jsonView";

export function createApp(model = new TodoModel()) {
  const controller = new ToDoController(model);

  return async function fetch(request: Request): Promise<Response> {
    const url = new URL(request.url);
```

```
const path = url.pathname;
```

Esse primeiro trecho prepara a composição principal. O model é criado por padrão, o controller é instanciado e o fetch handler passa a ser o ponto de entrada das requisições.

src/app.ts — rota raiz

```
if (request.method === "GET" && path === "/") {
  return json({ name: "bun-todo-api", status: "ok" });
}
```

A rota raiz funciona como um health check mínimo e também serve para mostrar o uso direto de Request e Response, sem framework adicional.

src/app.ts — coleção /todos

```
if (path === "/todos") {
  if (request.method === "GET") return controller.list();
  if (request.method === "POST") return controller.create(request);
  return error("Método não permitido", 405);
}
```

O segundo bloco trata a coleção /todos. Aqui a aplicação diferencia operações de leitura e criação e rejeita outros métodos com 405.

src/app.ts — item individual e fallback

```
const match = path.match(/^\/todos\/(\d+)\$/);
if (match) {
  const id = Number(match[1]);
  if (request.method === "GET") return controller.get(id);
  if (request.method === "PATCH") return controller.update(request, id);
  if (request.method === "DELETE") return controller.remove(id);
  return error("Método não permitido", 405);
}

return error("Rota não encontrada", 404);
};
```

Por fim, a expressão regular identifica rotas parametrizadas. O trecho também deixa explícito o fallback 404 para qualquer caminho não reconhecido.

4.7 Inicializando o servidor

O arquivo server.ts é curto, mas concentra decisões de inicialização: leitura da porta, binding em 0.0.0.0 e criação do servidor a partir do handler da aplicação.

src/server.ts — imports e configuração

```
import { createApp } from "./app";

const port = Number(process.env.PORT ?? 3000);
```

A porta é resolvida por variável de ambiente com fallback para 3000, o que facilita uso local e também containerização.

src/server.ts — Bun.serve

```
const server = Bun.serve({
  port,
```

```
hostname: "0.0.0.0",
fetch: createApp(),
});
```

A chamada a `Bun.serve` cria o servidor HTTP e associa o fetch handler retornado por `createApp`.

src/server.ts — log de inicialização

```
console.log(`API disponível em http://localhost:${server.port}`);
```

O log final é simples, mas suficiente para confirmar o endereço de execução durante os testes locais.

4.8 Endpoints

Método	Rota	Descrição	Resposta
GET	/	Health check simples	200
GET	/todos	Lista todos os itens	200
POST	/todos	Cria uma tarefa	201
GET	/todos/:id	Busca uma tarefa	200 / 404
PATCH	/todos/:id	Atualiza title e/ou completed	200 / 400 / 404
DELETE	/todos/:id	Remove a tarefa	204 / 404

4.9 Executando localmente

Modo de desenvolvimento com `--watch`

```
bun run dev

# Saída esperada:
# API disponível em http://localhost:3000
```

O arquivo `data/todos.sqlite` será criado no primeiro acesso ao model. Como o servidor escuta em `0.0.0.0`, ele funciona tanto para acesso local quanto dentro de um container; durante o desenvolvimento, use `localhost` no cliente.

CAPÍTULO 05

Testes: cURL e bun:test

Do teste manual do contrato HTTP a uma suíte rápida com banco em memória.

5.1 Testando com cURL

Com o servidor em execução, cURL é suficiente para verificar o contrato principal. Os exemplos abaixo usam sintaxe de shells Unix. No PowerShell, prefira `curl.exe` para evitar conflitos com aliases e adapte aspas quando necessário.

Criar e consultar

```
# Criar
curl -i -X POST http://localhost:3000/todos \
  -H "Content-Type: application/json" \
  -d '{"title": "Estudar Bun"}'

# Listar
curl -i http://localhost:3000/todos

# Buscar o id 1
curl -i http://localhost:3000/todos/1
```

Atualizar e remover

```
# Concluir a tarefa
curl -i -X PATCH http://localhost:3000/todos/1 \
  -H "Content-Type: application/json" \
  -d '{"completed": true}'

# Remover
curl -i -X DELETE http://localhost:3000/todos/1
```

5.2 Automatizando com bun:test

Bun inclui um test runner compatível com uma parte substancial da API de Jest. Ele reconhece arquivos `.test`, `.spec` e variantes equivalentes, suporta TypeScript, hooks de ciclo de vida, mocks, snapshots e modo watch. Para o exemplo, os testes exercitam o handler HTTP diretamente e criam um SQLite em memória para cada caso.

test/api.test.ts — setup da suíte

```
import { afterEach, beforeEach, describe, expect, test } from "bun:test";
import { Database } from "bun:sqlite";
import { createApp } from "../src/app";
import { TodoModel } from "../src/models/TodoModel";

describe("ToDo API", () => {
  let db: Database;
  let app: ReturnType<typeof createApp>;

  beforeEach(() => {
    db = new Database(":memory:");
    app = createApp(new TodoModel(db));
  });

  afterEach(() => db.close());
});
```

O setup prepara um banco em memória para cada teste e injeta um TodoModel próprio na aplicação. Isso evita interferência entre cenários e mantém a suíte rápida.

test/api.test.ts — teste de criação e listagem

```
test("cria e lista uma tarefa", async () => {
  const createResponse = await app(new Request("http://local/todos", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ title: "Estudar Bun" }),
  }));

  expect(createResponse.status).toBe(201);
  const created = await createResponse.json() as { id: number; title: string };
  expect(created.title).toBe("Estudar Bun");

  const listResponse = await app(new Request("http://local/todos"));
  const items = await listResponse.json() as Array<{ id: number }>;
  expect(items).toHaveLength(1);
  expect(items[0].id).toBe(created.id);
});
```

O primeiro caso verifica o fluxo principal: criar um item e confirmar sua presença na listagem. Assim, em um único teste, validam-se rota, controller, model e serialização JSON.

test/api.test.ts — teste de atualização

```
test("marca uma tarefa como concluída", async () => {
  const model = new TodoModel(db);
  const todo = model.create("Escrever testes");

  const response = await app(new Request(`http://local/todos/${todo.id}`, {
    method: "PATCH",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ completed: true }),
  }));

  expect(response.status).toBe(200);
  expect(await response.json()).toMatchObject({ completed: true });
});
```

O segundo teste concentra-se na atualização parcial. Ele prepara um item inicial e depois valida se PATCH consegue marcar a tarefa como concluída.

Comandos principais

```
# Executar toda a suíte
bun test

# Cobertura de código
bun test --coverage

# Reexecutar quando arquivos mudarem
bun test --watch
```

Teste sem porta de rede

Ao testar createApp diretamente, a suíte verifica roteamento, controller, view e SQLite sem depender de uma porta livre. Isso torna o teste rápido e previsível. Um teste end-to-end real, iniciando o servidor, pode ser acrescentado depois para validar comportamento de rede e encerramento do processo.

5.3 O que testar em seguida

POST sem title e JSON inválido, esperando HTTP 400.

GET e DELETE para ids inexistentes, esperando HTTP 404.

PATCH com completed em tipo incorreto.

Persistência em arquivo e reinicialização do processo.

Concorrência de leitura/escrita se a aplicação deixar de ser apenas didática.

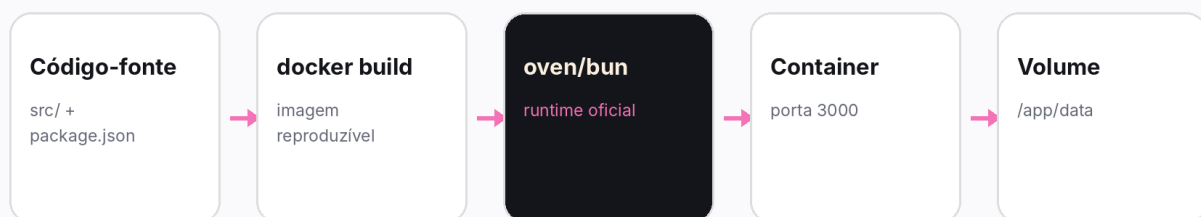
CAPÍTULO 06

Deploy simples com Docker

Empacotar o runtime é fácil; preservar o SQLite exige uma decisão explícita de volume.

Deploy com Docker: do código ao container

Para SQLite, o volume persistente é parte da arquitetura — não apenas um detalhe de execução.



Regra prática: o container pode ser descartável; o arquivo SQLite não deve ser.

Figura 5 — Fluxo de build e execução em container com persistência externa do diretório de dados.

6.1 Dockerfile

O Bun publica imagens oficiais com variantes Debian, slim, distroless e Alpine. Para manter o exemplo legível, o Dockerfile usa uma etapa de teste com a imagem completa e uma etapa final slim. Como a aplicação não possui dependências de runtime de terceiros, a imagem final precisa apenas do código-fonte e do próprio Bun.

Dockerfile — etapa de testes

```
FROM oven/bun:1 AS test
WORKDIR /app

COPY package.json bun.lock ./
RUN bun install --frozen-lockfile
COPY src ./src
COPY test ./test
RUN bun test
```

A primeira etapa instala dependências e executa a suíte. Em pipelines reais, isso ajuda a impedir que uma imagem quebrada siga adiante no processo de entrega.

Dockerfile — etapa de release

```
FROM oven/bun:1-slim AS release
WORKDIR /app
COPY --from=test /app/src ./src
RUN mkdir -p /app/data && chown -R bun:bun /app

USER bun
ENV PORT=3000
EXPOSE 3000
```

```
CMD ["bun", "src/server.ts"]
```

A etapa final é mais enxuta e prepara o diretório persistente de dados. O uso do usuário bun reduz privilégios e deixa a imagem mais apropriada para execução.

.dockerignore

```
node_modules
.git
.env
*.sqlite
*.sqlite-wal
*.sqlite-shm
coverage
```

O .dockerignore impede que arquivos locais desnecessários participem do contexto de build, reduzindo tempo de envio ao daemon e evitando surpresas com bancos locais ou caches.

6.2 Build e execução

Build e execução com volume nomeado

```
docker build --pull -t bun-todo-api .

docker run --rm \
  -p 3000:3000 \
  -v bun-todo-data:/app/data \
  bun-todo-api
```

O volume bun-todo-data é essencial porque o container deve ser tratado como descartável. Sem um volume ou bind mount, substituir o container também substitui o arquivo SQLite armazenado em sua camada gravável. Em produção, além de persistência, defina uma política de backup e valide como checkpoints e arquivos WAL são tratados pela infraestrutura.

6.3 Outras estratégias de distribuição

Docker é apenas uma das opções. A documentação do Bun descreve a geração de executáveis autocontidos com `bun build --compile`. Para produção, o guia recomenda opções como minificação, sourcemaps e, quando apropriado, bytecode. Esse formato reduz trabalho de resolução e transpile no startup e é especialmente interessante para CLIs, serviços pequenos e distribuição sem uma instalação separada do Bun.

Exemplo de executável compilado

```
bun build --compile --minify --sourcemap \
  ./src/server.ts --outfile bun-todo-api

./bun-todo-api
```

Em servidores tradicionais, a documentação também apresenta uso com PM2 como process manager. Frameworks como Next.js, SvelteKit, Nuxt, Hono e Elysia possuem guias específicos no ecossistema do Bun. Em plataformas que executam containers, a imagem Docker tende a ser o caminho mais portátil; em VPS, um processo Bun gerenciado por systemd ou PM2 é igualmente possível.

SQLite muda o desenho de deploy

Plataformas com filesystem efêmero ou múltiplas réplicas independentes exigem cuidado. SQLite é excelente para muitas cargas, mas não deve ser "escalado horizontalmente" copiando o mesmo arquivo entre instâncias sem uma estratégia própria de coordenação. Para esse cenário, considere storage persistente compatível, arquitetura single-writer ou um banco de rede.

CAPÍTULO 07

Próximos passos

O valor do Bun aparece quando o projeto cresce além do exemplo mínimo e o toolchain continua integrado.

Depois de construir a API deste manual, o caminho mais produtivo é aprofundar os subsistemas que realmente afetam o seu tipo de aplicação. Em vez de perseguir apenas números de benchmark, vale observar compatibilidade, segurança, comportamento em produção e custo operacional.

7.1 Segurança e cadeia de dependências

Estude `trustedDependencies` e a política de lifecycle scripts do `bun install` antes de liberar scripts de terceiros.

Revise a documentação de secrets, variáveis de ambiente e APIs de proteção disponíveis no runtime.

Mantenha Bun e dependências atualizados, mas fixe versões em CI/produção quando a reprodutibilidade for importante.

Trate o arquivo SQLite, backups e permissões de filesystem como dados de produção, não como parte descartável do container.

7.2 Benchmarks que fazem sentido

A documentação do projeto contém uma seção de benchmarking e publica resultados para componentes específicos. Use esses números como hipótese, não como conclusão. A melhor avaliação é reproduzir sua carga: startup, p95/p99 de rotas, consumo de memória, throughput com o banco real, tempo de instalação em CI e duração da suíte de testes. Quando comparar runtimes, mantenha framework, payload, hardware e versão das dependências constantes.

7.3 APIs e recursos para explorar

Área	O que aprofundar
Bun.serve / HTTP	Rotas, TLS, WebSockets, streaming e servidores full-stack.
bun:sqlite / SQL	Transações, prepared statements, WAL, serialização e API SQL unificada.
bun:test	Mocks, snapshots, DOM testing, cobertura e reporters.
Package manager	Workspaces, lockfile, trusted dependencies, catalogs e registries.
Bundler	Build para browser/server, plugins, HTML, minificação e executáveis.
Compatibilidade Node	APIs implementadas, diferenças conhecidas e migração incremental.
Runtime avançado	Workers, FFI, shell, file I/O, hashing, Redis e utilitários.

7.4 Quando usar Bun?

Bun é particularmente atraente quando a equipe valoriza inicialização rápida, uma CLI integrada, TypeScript sem etapa de configuração adicional e redução do número de ferramentas no projeto. Também é uma opção natural para experimentar APIs nativas como `bun:sqlite` ou para acelerar instalação e testes em projetos que ainda executam em Node.js.

A migração completa exige um critério um pouco diferente: a pergunta deixa de ser “Bun é rápido?” e passa a ser “todas as dependências, observabilidade, deploy e casos de borda do meu sistema se comportam como esperado?”. Em sistemas maduros, compatibilidade e previsibilidade valem mais do que qualquer microbenchmark isolado. A adoção incremental é uma forma sensata de obter os benefícios sem transformar a troca de runtime em uma reescrita.

Fechamento

O melhor primeiro projeto com Bun não precisa ser grande. Uma API pequena, testada e containerizada já permite observar o que diferencia o runtime: APIs web nativas, SQLite embutido, test runner integrado e uma experiência de linha de comando coerente.

Referências

- BUN. Bun Runtime. Bun Docs, 2026. Disponível em: <https://bun.sh/docs/runtime>. Acesso em: 16 ago. 2026.
- BUN. Containerize a Bun application with Docker. Guides, 2026. Disponível em: <https://bun.sh/guides/ecosystem/docker>. Acesso em: 16 ago. 2026.
- BUN. HTTP Server / Bun.serve. Bun Docs, 2026. Disponível em: <https://bun.sh/docs/runtime/http/server>. Acesso em: 16 ago. 2026.
- BUN. Installation. Bun Docs, 2026. Disponível em: <https://bun.sh/docs/installation>. Acesso em: 16 ago. 2026.
- BUN. Press kit: brand assets and colours. Bun, 2026. Disponível em: <https://bun.sh/press-kit>. Acesso em: 16 ago. 2026.
- BUN. Run Bun as a daemon with PM2. Guides, 2026. Disponível em: <https://bun.sh/guides/ecosystem/pm2>. Acesso em: 16 ago. 2026.
- BUN. SQLite. Bun Docs, 2026. Disponível em: <https://bun.sh/docs/runtime/sqlite>. Acesso em: 16 ago. 2026.
- BUN. Single-file executable. Bun Docs, 2026. Disponível em: <https://bun.sh/docs/bundler/executables>. Acesso em: 16 ago. 2026.
- BUN. Test runner. Bun Docs, 2026. Disponível em: <https://bun.sh/docs/test>. Acesso em: 16 ago. 2026.
- BUN. bun install. Package Manager Docs, 2026. Disponível em: <https://bun.sh/docs/pm/cli/install>. Acesso em: 16 ago. 2026.
- DENO. Get started with Deno. Deno Docs, 2026. Disponível em: <https://docs.deno.com/runtime/>. Acesso em: 16 ago. 2026.
- DOCKER. Docker documentation. Docker Docs, 2026. Disponível em: <https://docs.docker.com/>. Acesso em: 16 ago. 2026.
- NODE.JS. About Node.js. OpenJS Foundation, 2026. Disponível em: <https://nodejs.org/en/about>. Acesso em: 16 ago. 2026.
- NODE.JS. Modules: TypeScript. Node.js v26 Documentation, 2026. Disponível em: <https://nodejs.org/api/typescript.html>. Acesso em: 16 ago. 2026.
- SQLITE. Write-Ahead Logging. SQLite Documentation, 2026. Disponível em: <https://www.sqlite.org/wal.html>. Acesso em: 16 ago. 2026.
- SUMNER, Jarred. Bun is joining Anthropic. Bun Blog, 2 dez. 2025. Disponível em: <https://bun.sh/blog/bun-joins-anthropic>. Acesso em: 16 ago. 2026.
- SUMNER, Jarred. Rewriting Bun in Rust. Bun Blog, 8 jul. 2026. Disponível em: <https://bun.sh/blog/bun-in-rust>. Acesso em: 16 ago. 2026.

Nota sobre marcas e exemplos

Bun, Node.js, Deno, Docker e demais nomes citados pertencem a seus respectivos titulares. O logotipo do Bun utilizado na capa foi obtido do press kit oficial e não foi recolorido, em conformidade com a orientação de marca publicada pelo projeto.

Os exemplos de código deste manual são didáticos e voltados à introdução do runtime. Antes de usar qualquer trecho em produção, recomenda-se revisar segurança, observabilidade, tratamento de erros, políticas de backup, compatibilidade de dependências e aderência à arquitetura do sistema real.